HOBBY PROJECT CASE STUDY

Trend Finder

A personal product-research workspace bringing Google Trends and Reddit signals together to compare home-office ideas through ranked scans, visible progress, and saved history.

SYSTEM	SCOPE	APPROACH
Local Product Research Tool	**Home-Office Ideas Trends · Reddit · Rankings**	**Hobby Project Single-User · Local-First**

01 The Problem

Finding a home-office product idea worth exploring means looking beyond a single popular search term or an isolated online discussion.

Search interest and community conversations offer different perspectives. Moving between them manually makes it harder to compare candidates consistently, repeat the research, and remember how an idea looked in an earlier scan.

> Trend Finder brings candidate keywords, Google Trends direction, Reddit mentions, and scan history into one focused workspace for personal product exploration.

02 What Was Built

A connected research loop spanning configuration, signal collection, ranked comparison, and a closer look at each candidate.

RESEARCH DASHBOARD

Compare candidate keywords in a ranked list, inspect their component signals, and open a detail view with a saved-score history trace.

CONFIGURABLE RESEARCH

Manage product keywords and synonyms, choose up to five subreddits, and set a lookback window from one to twenty-four months.

CONNECTED SIGNAL COLLECTION

Collect Google Trends interest data and search Reddit posts and comments for keyword and synonym matches across selected communities.

LIVE SCAN WORKFLOW

Follow each keyword from pending to scanning to completion, then return to refreshed rankings backed by locally stored results.

03 How It Works

A React dashboard calls a local FastAPI service. The backend collects demand signals, calculates a relative score for each keyword, and saves scan results in SQLite.

React dashboard → FastAPI → Signal collectors

Google Trends + Reddit → Normalize → Rank → SQLite

Scan events → Live progress Saved runs → Rankings & history

Configure the research scope

Keywords and synonyms define the candidate list. Selected subreddits focus the discussion search, while a shared lookback setting keeps the collection window aligned across the two sources.

Collect search and community signals

The Google Trends collector compares recent interest with the earlier portion of the window. The Reddit integration uses PRAW to search selected communities and count distinct matching posts and comments, including synonym matches.

Normalize, combine, and persist

Signals are min-max normalized across the current keyword batch. The two-source score combines 60% Google Trends and 40% Reddit mentions on a 0–100 scale. Scan runs retain the raw signals and composite scores for later review.

Stream progress, then explore results

POST /scan streams per-keyword events as work progresses. The dashboard updates its scan panel, then loads GET /results to show the latest ranking and each keyword’s composite-score history.

04 Key Features

Ranked candidate comparison — a single dashboard orders home-office keywords by composite score and keeps the latest scan timestamp in view.

Google Trends direction — recent search interest is compared with earlier interest within the selected window to capture change rather than a single snapshot.

Integrated Reddit research — community searches connect keywords and synonyms to matching posts and comments, bringing discussion activity into the research workflow.

Editable research scope — add, rename, and remove keywords and synonyms; manage up to five subreddits; adjust the lookback window without changing code.

Live per-keyword progress — a scan panel tracks pending, scanning, completed, and failed keywords, with collected signal values and an overall progress indicator.

Signal-level detail — select a ranked candidate to inspect its raw Trends value, Reddit mention count, composite score, and score-based summary.

Saved scan history — local persistence feeds compact score traces and a larger detail chart, making earlier scan results accessible from the same dashboard.

Clear interface states — loading messages, empty-state guidance, error banners, and scan feedback make the current application state visible.

Focused local setup — React and FastAPI run as two local processes, with SQLite storage and seeded home-office keywords providing a practical starting point.

05 Technical Approach

The implementation separates the research interface, API workflow, source collectors, and local storage. Each layer has a narrow role in the scan-to-review journey.

LAYER	STACK	PURPOSE
Interface	React; TypeScript; Vite; React Router	Dashboard and settings routes with typed API data
Presentation	Tailwind CSS; inline styles; custom SVG charts	Signal-console interface, scan progress, and history traces
API & workflow	Python; FastAPI; Pydantic; Server-Sent Events	Configuration endpoints, scan orchestration, and live events
Signal collection	pytrends; PRAW; python-dotenv	Search-interest change and Reddit keyword / synonym matching
Persistence	SQLAlchemy; SQLite	Keywords, communities, settings, scan runs, and result history
Local execution	Uvicorn; Vite dev server; PowerShell launcher	Two-process local setup with automatic table creation and seeding

Design decisions that matter

Keep the research loop explicit. Configuration, collection, ranking, and inspection remain distinct steps, making the workflow easy to follow and revisit.

Compare candidates within a scan. Batch normalization puts search-interest change and mention counts onto a shared scale before they are combined.

Match the infrastructure to the project. A local SQLite database and a small two-process setup suit a personal tool without introducing hosted-service overhead.

06 Value Delivered

From scattered product-research signals to a repeatable personal exploration workflow:

Focused
RESEARCH SCOPE

Comparable
RANKED CANDIDATES

Repeatable
SCANS & HISTORY

- **One place to compare ideas.** — Google Trends direction and Reddit discussion activity sit alongside each candidate, reducing the need to assemble a shortlist across separate tools.

- **Control over what gets researched.** — Editable keywords, synonyms, communities, and lookback settings let the same workflow explore different home-office product themes.

- **A more visible research process.** — Per-keyword scan states and component-level readouts make it easier to follow collection and understand what contributes to a ranking.

- **A useful record to return to.** — Saved runs and score traces turn a one-off lookup into an ongoing hobby-project workflow: configure, scan, compare, and revisit.

Trend Finder — a local-first hobby project connecting configurable product research, Google Trends, Reddit signals, and saved scan history in one focused application.